

Imports

```
import pandas as pd
import csv
import json
from pprint import pprint
import transformers
import accelerate
from transformers import AutoTokenizer, AutoModelForCausalLM
from transformers import TrainingArguments, Trainer, DataCollatorWithPadding
import transformers
from transformers import logging as hf_logging
from tqdm import tqdm
import datasets
import torch, gc
import logging
```

```
df_raw = pd.read_csv('train.csv', delimiter=',')
df_raw.head()
```

	qtype	Question	Answer
0	susceptibility	Who is at risk for Lymphocytic Choriomeningiti...	LCMV infections can occur after exposure to fr...
1	symptoms	What are the symptoms of Lymphocytic Choriomen...	LCMV is most commonly recognized as causing ne...
2	susceptibility	Who is at risk for Lymphocytic Choriomeningiti...	Individuals of all ages who come into contact ...
3	exams and tests	How to diagnose Lymphocytic Choriomeningitis (...)	During the first phase of the disease, the mos...
4	treatment	What are the treatments for Lymphocytic Chorio...	Aseptic meningitis, encephalitis, or meningoen...

Comprehensive Medical Q&A Dataset from Kaggle: <https://www.kaggle.com/datasets/thedevastator/comprehensive-medical-q-a-dataset?resource=download>

```
df_raw.shape
```

```
(16407, 3)
```

Config Enviroment

```
logger = logging.getLogger(__name__)
global_config = None
```

```
device_count = torch.cuda.device_count()
if device_count > 0:
    logger.debug('Select GPU device')
    device = torch.device('cuda')
else:
    logger.debug('Select CPU device')
    device = torch.device('cpu')
```

```
gc.collect()
torch.cuda.empty_cache()
```

```
model = 'TinyLlama/TinyLlama-1.1B-Chat-v1.0'
```

Data Process

```
csv_file = 'train.csv'
jsonl_file = 'health.jsonl'
```

```
with open(csv_file, mode='r', encoding='utf-8') as f_csv, open(jsonl_file, mode='w', encoding='utf-8') as f_jsonl:
    reader = csv.DictReader(f_csv)
    for row in reader:
        data = {'question': row['Question'], 'answer': row['Answer']}
        f_jsonl.write(json.dumps(data) + '\n')
```

```
instruction_health_df = pd.read_json(jsonl_file, lines=True)
examples = instruction_health_df.to_dict()
```

```
print(f'Question: {examples["question"][0]} Answer: {examples["answer"][0]}')
```

```
Question: Who is at risk for Lymphocytic Choriomeningitis (LCM)? ? Answer: LCMV infections can occur after exposure to fresh urine, droppings, saliva, or nesting materials from infected rodents. Transmission may also occur when these materials are directly introduced into broken skin, the nose, the eyes, or the mouth, or presumably, via the bite of an infected rodent. Person-to-person transmission has not been reported, with the exception of vertical transmission from infected mother to fetus, and rarely, through organ transplantation.
```

```
prompt_template = """### Question:
{question}
```

```
### Answer: """
```

```
num_examples = len(examples['question'])
finetuning_dataset = []
```

```
for i in range(num_examples):
    question = examples['question'][i]
    answer = examples['answer'][i]
    text_with_prompt_template = prompt_template.format(question=question)
    finetuning_dataset.append({'question': text_with_prompt_template, 'answer': answer})
```

```
tokenizer = AutoTokenizer.from_pretrained(model)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
```

```
warnings.warn(
```

```
tokenizer_config.json: 1.29k/? [00:00<00:00, 116kB/s]
```

```
tokenizer.model: 100%
```

```
500k/500k [00:01<00:00, 363kB/s]
```

```
tokenizer.json: 1.84M/? [00:00<00:00, 61.3MB/s]
```

```
special_tokens_map.json: 100%
```

```
551/551 [00:00<00:00, 34.3kB/s]
```

```
pprint(finetuning_dataset[0])
```

```
{'answer': 'LCMV infections can occur after exposure to fresh urine, '
'droppings, saliva, or nesting materials from infected rodents. '
'Transmission may also occur when these materials are directly '
'introduced into broken skin, the nose, the eyes, or the mouth, or '
'presumably, via the bite of an infected rodent. Person-to-person '
'transmission has not been reported, with the exception of vertical '
'transmission from infected mother to fetus, and rarely, through '
'organ transplantation.',
'question': '### Question:\n'
'Who is at risk for Lymphocytic Choriomeningitis (LCM)? ?\n'
'\n'
'### Answer:'}
```

Tokenization

```
def tokenize_function(examples):
    # Concatena pergunta + resposta, pois o modelo é causal
    if 'question' in examples and 'answer' in examples:
        text = [
            q + a for q, a in zip(examples['question'], examples['answer'])
        ]
    else:
        text = examples['text']

    # Pad_token = eos_token (TinyLlama não tem pad_token por padrão)
    tokenizer.pad_token = tokenizer.eos_token

    # Tokenização padronizada
    tokenized_inputs = tokenizer(
        text,
        return_tensors='pt',
        padding='max_length',
        truncation=True,
        max_length=2048
    )

    # Labels devem ser iguais aos input_ids para causal LM
```

```

tokenized_inputs['labels'] = tokenized_inputs['input_ids'].clone()

return tokenized_inputs

```

```
finetuning_dataset_loaded = datasets.load_dataset('json', data_files=jsonl_file, split='train')
```

Generating train split: 16407/0 [00:00<00:00, 58288.36 examples/s]

```

tokenized_dataset = finetuning_dataset_loaded.map(
    tokenize_function,
    batched=True,
    batch_size = 1,
    drop_last_batch = True
)

```

Map: 100%

16407/16407 [01:09<00:00, 182.20 examples/s]

```
print(tokenized_dataset)
```

```

Dataset({
  features: ['question', 'answer', 'input_ids', 'attention_mask', 'labels'],
  num_rows: 16407
})

```

```

split_dataset = tokenized_dataset.train_test_split(test_size=0.2, shuffle=True, seed=123)
print(split_dataset)

```

```

DatasetDict({
  train: Dataset({
    features: ['question', 'answer', 'input_ids', 'attention_mask', 'labels'],
    num_rows: 13125
  })
  test: Dataset({
    features: ['question', 'answer', 'input_ids', 'attention_mask', 'labels'],
    num_rows: 3282
  })
})

```

```

train_dataset = split_dataset['train']
test_dataset = split_dataset['test']

```

Training Model

```
base_model = AutoModelForCausalLM.from_pretrained(model)
```

config.json: 100%

608/608 [00:00<00:00, 30.7kB/s]

model.safetensors: 100%

2.20G/2.20G [00:25<00:00, 208MB/s]

generation_config.json: 100%

124/124 [00:00<00:00, 11.1kB/s]

```
base_model.config.use_cache = False
```

```

dataset_path = f'/content/{jsonl_file}'
use_hf = False

```

```

training_config = {
    'model': {
        'pretrainend_name': model,
        'max_length': 2048
    },
    'datasets': {
        'use_hf': use_hf,
        'path': dataset_path
    },
    'verbose': True
}

```

```

def inference(text, model, tokenizer, max_input_tokens = 1000, max_output_tokens=100):
    input_ids = tokenizer.encode(
        text,
        return_tensors = 'pt',
        truncation=True,

```

```

        max_length = max_input_tokens
    )
    device = model.device
    generated_tokens_with_prompt = model.generate(
        input_ids = input_ids.to(device),
        max_length = max_output_tokens
    )
    generated_text_with_prompt = tokenizer.batch_decode(generated_tokens_with_prompt, skip_special_tokens=True)
    generated_text_answer = generated_text_with_prompt[0][len(text):]

    return generated_text_answer

```

```

test_text = test_dataset[1]['question']
print("Question input (test):", test_text)
print(f"Correct answer docs: {test_dataset[1]['answer']}")
print("Model's answer: ")
print(inference(test_text, base_model, tokenizer))

```

Question input (test): Is arginase deficiency inherited ?
 Correct answer docs: This condition is inherited in an autosomal recessive pattern, which means both copies of the gene in each cell have mutations.
 Model's answer:

Setup Training

```
max_steps = 3
```

```

trained_model_name = f'{model}_{max_steps}_steps'
output_dir = trained_model_name

```

```

training_args = TrainingArguments(
    learning_rate = 1.0e-5,
    num_train_epochs = 3,
    max_steps = max_steps,
    per_device_train_batch_size = 4,
    output_dir = output_dir,
    overwrite_output_dir = False,
    disable_tqdm = False,
    eval_steps = 120,
    save_steps = 120,
    warmup_steps = 1,
    per_device_eval_batch_size = 1,
    eval_strategy = 'steps',
    logging_strategy = 'steps',
    logging_steps = 1,
    optim = 'adafactor',
    gradient_accumulation_steps = 4,
    gradient_checkpointing = True,
    load_best_model_at_end = True,
    save_total_limit= 1,
    metric_for_best_model = 'eval_loss',
    greater_is_better = False,
    report_to='none'
)

```

```
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

```

trainer = Trainer(
    model = base_model,
    args = training_args,
    train_dataset= train_dataset,
    eval_dataset= test_dataset,
    tokenizer=tokenizer,
    data_collator=data_collator
)

```

```

/tmp/ipython-input-1844986043.py:1: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0
trainer = Trainer(

```

```
training_output = trainer.train()
```

The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config [3/3 09:10, Epoch 0/1]

Step Training Loss Validation Loss

```
save_dir = f'{output_dir} / final'
trainer.save_model(save_dir)
print(f'Saved model: {save_dir}')
```

Saved model: TinyLlama/TinyLlama-1.1B-Chat-v1.0_3_steps / final

Assessment

```
finetuned_model = AutoModelForCausalLM.from_pretrained(save_dir, local_files_only=True)
finetuned_model.to(device)
```

The module name final (originally final) is not a valid Python identifier. Please rename the original module to LlamaForCausalLM(

```
(model): LlamaModel(
  (embed_tokens): Embedding(32000, 2048, padding_idx=2)
  (layers): ModuleList(
    (0-21): 22 x LlamaDecoderLayer(
      (self_attn): LlamaAttention(
        (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
        (k_proj): Linear(in_features=2048, out_features=256, bias=False)
        (v_proj): Linear(in_features=2048, out_features=256, bias=False)
        (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
      )
      (mlp): LlamaMLP(
        (gate_proj): Linear(in_features=2048, out_features=5632, bias=False)
        (up_proj): Linear(in_features=2048, out_features=5632, bias=False)
        (down_proj): Linear(in_features=5632, out_features=2048, bias=False)
        (act_fn): SiLUActivation()
      )
      (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
      (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
    )
  )
  (norm): LlamaRMSNorm((2048,), eps=1e-05)
  (rotary_emb): LlamaRotaryEmbedding()
)
(lm_head): Linear(in_features=2048, out_features=32000, bias=False)
)
```

```
test_text = test_dataset[1]['question']
print("Question input (test):", test_text)
original_answer = test_dataset[1]['answer']
print(f"Correct answer docs: {original_answer}")
```

Question input (test): Is arginase deficiency inherited ?
Correct answer docs: This condition is inherited in an autosomal recessive pattern, which means both copies of th

```
test_text = f"### Question:\n{test_dataset[1]['question']}\n\n### Answer:"
print("Model's answer:")
generated_answer = inference(text=test_text, model=finetuned_model, tokenizer=tokenizer)
print(generated_answer)
```

The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a cons
Model's answer:

Yes, arginase deficiency is inherited in an autosomal recessive manner. This means that to have the condition, bo

```
def is_exact_match(a, b):
    return a.strip() == b.strip
```

```
finetuned_model.eval()
```

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(32000, 2048, padding_idx=2)
    (layers): ModuleList(
      (0-21): 22 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
          (k_proj): Linear(in_features=2048, out_features=256, bias=False)
          (v_proj): Linear(in_features=2048, out_features=256, bias=False)
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=2048, out_features=5632, bias=False)
          (up_proj): Linear(in_features=2048, out_features=5632, bias=False)
          (down_proj): Linear(in_features=5632, out_features=2048, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((2048,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=2048, out_features=32000, bias=False)
)
```

```

    )
    (mlp): LlamaMLP(
      (gate_proj): Linear(in_features=2048, out_features=5632, bias=False)
      (up_proj): Linear(in_features=2048, out_features=5632, bias=False)
      (down_proj): Linear(in_features=5632, out_features=2048, bias=False)
      (act_fn): SiLUActivation()
    )
    (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
    (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
  )
  (norm): LlamaRMSNorm((2048,), eps=1e-05)
  (rotary_emb): LlamaRotaryEmbedding()
)
(lm_head): Linear(in_features=2048, out_features=32000, bias=False)
)

```

```

exact_match = is_exact_match(original_answer, generated_answer)
print(f"Exact Match: {exact_match}")

```

Exact Match: False

```

n = 10
metrics = {'exact_matches': []}
predictions = []
example_test_dataset = test_dataset.select(range(10))

```

example_test_dataset

```

Dataset({
  features: ['question', 'answer', 'input_ids', 'attention_mask', 'labels'],
  num_rows: 10
})

```

```

for i, item in tqdm(enumerate(example_test_dataset)):
    question = f"### Question:\n{item['question']}\n\n### Answer:"
    answer = item['answer']

    try:
        predicted_answer = inference(question, finetuned_model, tokenizer)
    except:
        continue
    predictions.append([predicted_answer, answer])

    exact_match = is_exact_match(predicted_answer, answer)
    metrics['exact_matches'].append(exact_match)

print(f'Number of exact matches: {sum(metrics['exact_matches'])}')

```

10it [00:18, 1.89s/it]Number of exact matches: 0

```

df_predictions = pd.DataFrame(predictions, columns=['predicted_answer', 'target_answer'])
df_predictions.head()

```

	predicted_answer	target_answer	
0	\nUrinary Incontinence in Men is a condition w...	The prostate is a walnut-shaped gland that is ...	
1	\nYes, arginase deficiency is inherited in an ...	This condition is inherited in an autosomal re...	
2	\n1. Treatment for Parasites - Paragonimiasis ...	Paragonimus infections are treatable by your h...	
3	\nCongenital muscular dystrophy is a genetic d...	Congenital muscular dystrophy (CMD) refers to ...	
4	\nSideroblastic anemia is a type of anemia tha...	Sideroblastic anemia is a heterogeneous group ...	

Próximas etapas:

[Gerar código com df_predictions](#)

[New interactive sheet](#)

